

Lec 05 Convolutional Neural Network - Part A

Lec 05 Convolutional Neural Network - Part A

Lecture	05 / 63 + intro
Duration	47:25
Captions	English (auto-generated)
Words	7,420
Video	SoKjqOB0YPw
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome to the lecture series on fundamentals of generative AI and the large language models. In this particular uh session, we will understand how exactly a computer interpret an image. Why not use fully connected neural networks for processing an image data? Then we will understand uh the convolutional neural networks. So, before we go in detail into the architecture of a CNN, we will understand what are the key components of a CNN. Then we will understand the first layer, that is convolutional layer, in detail in this particular session.

[00:40] So, when we look into this particular image, so for a human uh eye, it is an image. But how exactly a system understands an image? When you look closer into this, when we look here, there is a small box. And what exactly that box represents? It is The box is called a pixel. And every pixel, each pixel in an image is represented by a numerical value. A pixel here is a numerical value. Now, and once again, when we look into an image, one pixel does not contribute anything, but collection of such pixels will be able to understand what exactly is that particular object. For example, if you look into this, here we have a sun. So, one particular pixel will not give any information to us. But when we

[01:32] consider the collection nearby pixels, we'll be able to understand that it is a sun. So, what is that we are understanding of from this? Here an image can be represented as the matrix of pixel values. And what does each pixel indicates? It gives us the intensity of the pixel, which is a brightness, or it can be a color intensity when we are dealing with RGB. So, this is how an image is read by a system. Now, one more important information that we should know before we get into the architecture is whenever we look into an image, we have n number of rows and then the columns. So, here rows we call it as a height and then the columns we call it as width, which we call it as a dimension. So, a dimensions

[02:26] of an image can be defined as the number of pixels along its height and then the width. This one information is needed to us before we get into further details. So, next the most important we need to understand why not use a fully connected neural networks when we are dealing with an images. So, whenever we are when we discuss about the multi-layer perceptron, how is that we actually look into it? Multi-layer perceptron needs input as an independent features. It look each of the feature as an independent feature. And it always requires a data into a single vector.

[03:11] Now, if it requires a data in a single vector form, and if it treats each of the feature independently, then what are the challenges that we face it? Can we let's look into it. Now, suppose if we have an image and if that image is of a size 28 cross 28 into three. Now, what exactly is 28 28? We have seen sometime back, it is the dimension of an image, which is the height and then the width. And here three is a number of channels.

[03:44] Here three indicates we are dealing with an a colored image, that is RGB image. Now, if this is a one image, if you have to pass this into an a multi-layer perceptron, we have to convert that into an a single vector. So, what is the total size here? When we multiply, we get 2 3 5 2, which means the input size here is 2 3 5 2. We have these many number of features that we are passing it to into a multi-layer perceptron. But, when we look into this image, 28 28 3, is it not a very low-dimensional image it is? Yes, we will not be able to understand what exactly are the contents in this particular image.

[04:29] So, when we increase the dimension, suppose if you increase the dimension to 200 200, and once again when we consider an a colored image, then the size of this particular will be 120,000. Now, is it not something very huge? So, this is the size of an input that we are passing it into an a multi-layer perceptron. So, this is the reason why, you know, we cannot use multi-layer perceptron when we are dealing with images. Let us get a little more into detail what exactly is the problem. So, let us look into this.

[05:07] Now, what exactly we have discussed, we have seen that the image is represented in the form of an matrix of a pixel values. That is how the system reads

an image. Now, if we have to pass this to a single layer multi-layer perceptron, then what is the first step here? The image, which is in the you know, in a two-dimensional matrix, has to be converted into a single vector. So, if you convert that into a single vector, what exactly are the issues? First, we are going to lose the spatial relationship. It is going to destroy the spatial relationship. What exactly mean by this?

[05:52] Now, if you look into this particular image, here we have a curve. Here we have a curve and we have a line here. So, this is how an image eight is written. In this, every of the pixel, there is a relation with its neighboring pixel. And every pixel is positioned in a particular place. Now, when we convert that into a single vector, we are going to lose its relationship. The spatial relationship is being lost. That is how pixels are positioned and how they are related to each other.

[06:32] And that is the first point. And the second here, fully connected neural network treats every pixel as an independent feature. Of course, yes. Every feature is considered as an independent feature. Here, what is a feature in an image? Every of the pixel values are exactly are the features. Now, if we treat every pixel as an independent feature, then that is not correct when we are dealing with an images. Why? Because in an image, a single pixel does not contribute much.

[07:07] We will not get much information just by looking at a single pixel. But when we look into its neighboring pixels, collection of such neighboring pixel, we'll be able to understand what exactly is that particular object. Yeah. So, fully connected neural network treats every pixel as an independent feature. That cannot be possible when we are dealing with an images. And the third point here is local dependencies are lost. What do you mean by local dependencies is once again you see here, this is an edge. This is one curve. This is one line. So, when you convert that into a single vector, that particular information will be lost. That is the main problem why we cannot go we cannot use images for an

[07:54] uh when we are dealing with images, we cannot use a multi-layer perceptron. So, this is what has been explained. So, here the image is converted into image is always a seen in the form of an matrix of pixel values. Let us assume this is a matrix of a pixel values. When we convert that into an single vector, this is how the single vector will be represented. Now, what exactly is the problem? When you look into this particular matrix, here the horizontal neighbors are for one, uh two is a neighbor. For two, one and three are the neighbors. So, the horizontal neighbors are 1 2 3, 4 5 6, and 7 8 9. Similar way, the vertical neighbors are for four, the neighboring pixel is one and the seven. So, 1 4 7 um

[08:44] it is a vertical they are the vertical neighbors. But, when they convert that into a single vector Look into this. When we convert this into a single vector, here two and five are related to each other. But, you see two is here and five is here. So,

definitely the pixel positions are not together. Similar way, here three and four are together. But, three is here and four is here. So, this is what is exactly losing the spatial relationship.

[09:19] And next When this is a very small matrix that we have considered, which is just a 3 cross 3, but in general, when we consider 28 cross 28, let's take this example only. So, 28 cross 28, we have a 784. Now, when we have the input size as a 784, now just imagine when we have a multiple hidden layers. And since it is a multi-layer perceptron, one neuron will be connected to every other neuron. So, when we do in this a particular way, is it not we will have a lot many number of a parameters? What do you mean by parameters here? We will have so many weights because every neuron is connected to every other neuron. On every connection we have a weights. So, because of this the parameters there is an exploration in the

[10:11] parameters. When we have so many of such parameters, obviously what it is going to have an impact? The impact will be the training will the training the speed will be reduced. And thus, it increases the overfitting problem. This is the main reason why we cannot use multi-layer perceptron when we are dealing with an images. If that is the case, we definitely require one more neural network or one more deep learning model. That deep learning model here is a convolutional neural network.

[10:47] In short, we call it as a CNN. Now, what exactly is a CNN used for? CNN is mainly used whenever we are dealing with an images. When the data is an image, then the most appropriate architecture that will be using is CNN. So, what exactly is a CNN here? CNN is a specialized deep neural network mainly used for processing the data, specially when we are working with images. Okay. If this is the case, we understood that for images we require CNN model. Fine.

[11:24] So, what all the applications? For what all applications we can use CNN? We can go for CNN whenever we have an application such as image classification. Now here we have a sheep. So when we look into this particular whether it is a sheep or not. So one more example if you have to take so whether a particular image is a benign image or a malignancy image whether particular image is a cancerous one or non-cancerous. So for a classification application we can definitely use a CNN. And the second application where we can use a CNN is objective detection.

[12:07] Now what exactly is objective detection? In an image when we have a multiple objects. So you need to identify where exactly a particular object is present. Then we can such applications are objective detection applications. And here we use a CNN based models only. » [snorts] » And next we have an image segmentation. Now what exactly is image segmentation? We can see here one is a sheep and second one is a lamp. So here we are trying to identify where exactly the particular object is present. And we are interested only on that particular region only region of interest only. So

it is image segmentation. One more example if you have to understand now let's go back to a medical application. Now in an image

[13:02] where we have considered let us assume we have considered a CT scanned image and we have considered a CT scanned image of an ovarian region. In that the CT scan only in a particular region we have a tumor present. Now when we pass the entire image to a model is it not unnecessarily the model has to learn so many features which are really not contributing to us. In that case, can we just segment only that particular region and make a rest of the portion as a background so that the model need not have to focus on the remaining part and it can focus only on a particular region. So, this is what is an image segmentation. So, for an image segmentation also, we'll be using a CNN based model only.

[13:51] So, these are some of the applications where CNN can be used. It is just not for an classification. We can also go for an object detection and for an image segmentation. So, there is one more example here. So, this is a classification and the localization. So, all these belongs to a same class that is a sheep. You're just not classifying, you're also identifying where exactly this particular object is present. And this is object detection where there are a multiple objects that are present.

[14:25] » [snorts] » This is semantic segmentation. What is semantic segmentation here? When we look into this particular image, we have road, we have a grass and then we have all these animals are sheep. So, since we have a three different objects that are represent, each of the object is being a segmented with the similar pixels or similar color. So, here the road, this entire is a road so we are highlighting with one. This entire is a grass and this is a sheep.

[14:57] Then, we have something called as instance segmentation. What is instance segmentation? When we look into this entire sheep belongs to a same class. Therefore, all of this is being identified with the similar. But when it comes to instance, this is a sheep one, this is sheep two, this is sheep three. So, even though they all belongs to a same class, we are each of that are once again a different you know individual animal. Therefore, this is called an instance segmentation.

[15:28] So, all of this can be can be performed using a CNN based model. So, this is what the CNN is used to for. If this is the case, now we know that neural networks are perceptron is being built from the inspiration of biological neural system. If that is the case, how CNN has been built? What is the inspiration for building a CNN model? So, the inspiration is a human visual system. So, how a human eye will see an object?

[16:00] How human eye understands that particular object or an image? So, CNN is being built based on a human visual system. So, how exactly human visual system works? Whenever we look into any image, the very first thing that we understand from that image is we first see the outlines. We see the lines, then edges, corners. Then again, when we look into, we will start connecting those lines and we form an

object. And once we form an object, we try to identify that it is so and so. So, this is how a human visual system works. So, exactly CNN also mimics the human visual system.

[16:46] Where the earlier layers in the CNN model will focus on the local regions. That is, it first focuses on understanding the lines, edges, corners. Gradually, as we move deeper into the architecture, it starts understanding the image as a whole. So, this is what is an example. Earlier layers will focus on the lines. So, if you see, there are lines. These lines can be a horizontal line, vertical line, corners. Then, as we go deeper into the architecture, we can see that it is understanding an image as it is able to identify what exactly is this particular object.

[17:30] So, how human visual system works is similar way, even CNN also works. Yeah. So, before we get into the architecture, let us look into what are the key components of a CNN. So, the key components definitely we require an input and that input we always take consider an image. So, the very first component is a convolutional layer. What exactly is convolutional layer responsible for? It is mainly responsible for extracting the features.

[18:06] So, if you see here, the initial layer of convolutional the earlier layers will be focusing on a low level of features. Then, the middle level convolutional layers will focus on the later layers will be focusing on the middle level of features and the last layers of convolutional focus on high level of features. So, the the very first component in CNN is a convolutional layer. And next, we have an activation function and that activation function and more specifically, we will be using a ReLU.

[18:42] But, however, it is not always that we'll be using a ReLU. We'll see that in our hands-on session. So, the second component is an activation function. Third is a pooling layer. So, what exactly is a pooling layer responsible for? Ultimately, we will be passing the features to an you know, fully connected neural network only because fully connected is responsible for your classification or whatever the operation that we want to.

[19:13] But directly image cannot be passed to a fully connected. Why image cannot be passed? Because image size is more. When the size is more, large number of features will be passed as an input. So in order to avoid, we are first doing a convolution layer where we are first extracting the features. Then we are passing it to an activation function. Activation function will eliminate all those features which it does not contribute much. But we have still not reduced the size. What exactly pooling layer is responsible for?

[19:47] Pooling layer is mainly used for a downsampling. What is a downsampling here? It is going to reduce the size of the feature maps. Although all these are terminologies may be new for those who are listening those who are learning CNN for the first time, but don't worry. As we move further, you'll be understanding each

of these terms in detail. For now, this particular slide, try to understand what all different components.

[20:14] So the third component here is a pooling layer. And the fourth component is a fully connected layer. So what exactly is a fully connected layer? Definitely, we have to convert whatever the output of a pooling layer into a single vector. And that single vector will be passed to a fully connected layer which is mainly responsible of a classification. So the fourth component is a fully connected layer. Apart from this, there are optional layers. These are hyper parameter, you know, regularization techniques. And that is normalization and dropout. So these two are optional.

[20:56] But otherwise, whenever we are dealing with the CNN, all these are four are the important components. Convolutional layer, activation function, pooling layer, and fully connected layer. Okay. So, before we get into and first component, that is convolutional layer, I want you to look into this image and the first image and then understand how our human visual system works. So, that you can able to connect how convolutional layer you know the first layer, convolutional layer works. When we look into this particular image, when we see here » [snorts] » for the first time when you look into this image, what is that you are trying to extract? Is it not your first seeing all the outlines? So, by looking at the

[21:44] outline, you to some extent you are able to identify what exactly is this object. After that, again you start to connecting all these initial features, that is the outlines. You will connect these multiple outlines together and then you will try to form an shape. And by combining multiple shapes together, you are able to identify that I think it is an elephant. This is how you will be able to identify. So, initially you will be able to extract all of these features. This is how we will be able to extract. Then once we extract this, we are going to combine all of these you know the features that you have extracted.

[22:29] And then you will also be looking into the texture, the color, and then you'll be able to identify that it is an elephant. So, you are extracting the features. To extract the features, what has helped you? Is it not the human eye? The human eye has gone through the entire image, you know, and you you tried to extract the features. You have combined the features at the back end and you were able to identify an object. Similar way, here the very first layer is a convolutional layer.

[23:05] What exactly is a convolutional layer? This is a fundamental building block of a CNN mainly responsible for extracting the features from an input image. Now, how human eye was responsible to go through the entire image to extract the features, here also we require one eye and that eye we call it as a kernel. And what is the role of a kernel here? The kernel will go through the entire image and then it will extract the features.

[23:36] Now, if that is the case, whatever the values that we are seeing in the kernel,

are these values fixed? The answer is no. These are set of learnable you know filters. All these values we are going to learn over a period of time. During back propagation, you're going to update these values. So, what is convolution? It is an fundamental building block which is mainly used to extract the features from an input image. And how are we extracting the features? With the help of a kernel. And kernel here is since image is a matrix, definitely the kernel also will be a matrix.

[24:18] Now, if it is a matrix, we will have a values. And whatever the values that we are considering here, these are these values we are going to learn over a period of time. Now, we have understood that in order to extract the features we require a kernel. Now, how do it extracts how the features gets extracted? So, how it is? We know that from the beginning in the beginning of this session we have understood that we take an image and that image is an it is an input and image is always represented as an matrix of pixel values. Let us assume these are the matrix of pixel values. Then, we require an a kernel.

[24:59] So, the kernel is also called a filter. So, we'll be often interchanging these words, but it is one and the same. If I use a kernel, if I say kernel, it is a filter. Um you can understand it as a filter. When I say it's a filter, it is understood that it's a kernel. So, we'll be interchanging these. Uh so, uh this kernel, what we are going to do, since it is a matrix, definitely we'll be taking a size of a kernel definitely smaller than an input size.

[25:28] And here, we always uh go for try to use an odd-size kernel. So, what is odd-size kernel? It is 1 cross 1 or it is a 3 cross 3. It is just not only this. For this particular example, I I will first start with a 3 cross 3. Later, I'll tell why 1 cross 1 is not very much advisable. » [snorts] » So, now when we take a 3 cross 3, is it not a 3 cross 3 matrix here? So, this 3 cross 3 matrix, when it is when you want to perform an operation onto this input image, so definitely you will consider only 3 cross 3 part of this input image.

[26:07] So, now we will apply this. And the next, what is that operation we are going to do? We are going to perform a convolution operation. What is convolution operation? It is element-wise multiplication. One gets multiplied with this one. Zero gets multiplied with the zero. One gets multiplied with this one. Then, so it is an element-wise multiplication. Or you can consider it as one gets multiplied with this one. Zero gets multiplied with the zero. And one gets multiplied with this one. Then, zero gets multiplied with So, you will have an element-wise multiplication. And then, you will do a summation in order to create a feature map. So, now whatever we have seen so far, we will take Now, once we perform

[26:55] this entire operation, whatever the resultant matrix that we are going to get, that resultant uh matrix This is the resultant matrix. So, it is a feature map. We call it as a feature map. So, let's look into in detail here. We have an input image, which

is a 6 cross 6. We have considered an a kernel, which is of size a 3 cross 3. So, what is the first step? The first step is that the 3 cross 3 matrix of a kernel is applied onto the 3 cross 3 part of an input image. And what is that we are going to do? We have to first extract the features. To extract the features, we have to perform a convolution operation.

[27:37] What is convolution operation? It is an element-wise multiplication. So, can you see here? 3 is a multiplied with this one. Then, one is a multiplied with this one. Then, two is a multiplied with this one. So, this is what is the first one. Second, when you look into the zero gets multiplied with this zero. Then, we have a five multiplied with this zero. Then, seven multiplied with this zero. Then, one multiplied with minus one, eight multiplied with minus one, and two gets multiplied with this minus one. So, this is what is an element-wise multiplication, and you will do a summation. So, when we do a summation, we get minus five.

[28:22] So, this minus five is a resultant of this first operation. Then, what exactly we have done? This is a convolution operation. Whatever we are seeing, it's a convolution operation. And what is the you know, definition? We perform element-wise a multiplication and sum the result. Now, what exactly we have done? The I The kernel here has a seen this part of an image. With this, can we say that the image the complete image has been considered?

[28:54] Answer is no. So, now what you're going to do, you're going to stride it. You're going to move the kernel little towards the next portion of an image. When we look into this portion of an image, what we are getting now is this part of an image. We are looking into this part of an image. So, observe this a three. Everybody just observe this three. So, when you look into this part of an image, again we will do an element wise multiplication.

[29:23] Now, we are considering this part of an image and you will multiply with this kernel and you get this result. Again, we stride it. We now look into this part of an image and just to have a look at the three once again. No need to consider, just have a look at it. Just see the number three there. So, we are now considering this part of an image. Again, we are doing element wise multiplication and we get this result. And we continue this. We perform the entire We go through the entire We apply the kernel. We multiply.

[29:58] Finally, we apply the kernel onto this part of an image. We multiply and we get this. Right? So, by looking into by What exactly we have performed? We have taken an image. We have applied a kernel. We have passed the kernel through the entire image. We have extracted the features. Now, in this process, there is a something which I had asked you to look into this a three. So, why exactly I have asked you to focus on three is, let's go back and then I'll explain you this. Now, we are looking into this part of an image.

[30:35] So, which means we have considered this three also. After we perform the entire convolution operation, we have a stride it. When we have a stride, is it not we are again looking into this entire image for the second time? Earlier, it was this. Now, again we are looking into this. Okay. Then, you stride it stride, we are considering this part of an image again. Stride it, we are doing this. Then, when we come to when we stride and come down, we are looking into this portion once again.

[31:14] But, when we see three, three we have looked into only once. Similar way, it is just not applicable only to three, it is applicable to all the borders. Wherever we have a borders, this how three is looked only once, four also will be looked only once. Similar way, nine, similar way, two. Okay. What exactly we why have asked you to look into, we'll see that shortly. I hope convolution operation is clear to the learners here. So, what exactly we the important step here is you have to extract the features. So, we have extracted the features. What is mainly responsible for extracting the features here? It is the kernel. So, one example if you have to see here, we have an input image, we have a kernel, and by

[32:01] applying the kernel, you have extracted the feature. Now, just by looking one time, you may not be able to extract all the features. Can I again apply the same kernel this kernel onto this entire input image and extract the features? What is your thought process on this? We cannot do it, right? Why we cannot do? Because it is the same kernel. The same kernel when you go through the entire image, again it will be the same values, right? So, therefore, can I use one more kernel with different values and this kernel when it is applied onto this input image, is it not we get one more feature map?

[32:47] We get a different feature map where those values will be different from these values. Definitely answer is yes. So, which means when we use a different kernel, you see here now we have used a different kernel. Earlier the kernel was this. Now we are using a different kernel. It has extracted different features. So, more different kernels you use, different features gets extracted. This is what is the convolution operation.

[33:16] Let's quickly get into the math. What exactly is the math behind? What will be the output size? Here we have seen the output size is 4 cross 4. On what basis we are getting this 4 cross 4? The how how is that we calculated the you know, the feature map size? This is the input. That is something which is already known to us. Kernel is something that we are considering. So, we know the input size. We know the kernel. So, then we should know what will be this feature map size. Here the feature map size is 4 cross 4. How did we get this a feature map? What is the mathematical formula? The Ideally the mathematical formula is $n - f + 1$ divided by $s + 1$.

[34:01] into $n - f + 1$ divided by $s + 1$. So, this is what is the formula. Now, what

s stands for a stride. So, how much you want to move how you want to move. If the stride is 1, so is it not denominator is 1? Then is it not the formula is just n minus f plus 1 and n minus f plus 1? Is it not just the formula is so much? That is what is been written here. Just substitute the values. n value is 6. f value is a $3 + 1$. So, when we perform this calculation, we get to 4.

[34:37] So, this is one is the height, second is width. So, this is what is the feature map height and width. So, it is 4 cross a 4. Now, if you look into this, there is one a problem which has occurred. » [snorts] » The input size is 6 cross 6, then the feature map what we got is a 4 cross 4, which means some information is lost, right? There is a reduction in the size. If there is a reduction in the size, there is some information which has been uh lost. Correct? So, uh how is that we are going to avoid this?

[35:11] The uh solution is a padding. So, let us look into what exactly is a padding. So, why padding is required during convolution operation, the size of the output of feature map is mainly determined by the input, then we have a kernel. By applying kernel onto this input image, we get this a feature map. Now, the problem with this a feature map is the size has reduced. Now, because the size has uh you know, the you have got a smaller than the original input. Now, if it has if you if the size has reduced, it means that information is lost. Now, which information is lost? The information at the borders have been lost. But, what if there is some important information even in the border also? Now, say for

[36:00] example, in medical application, every part of an image is important. So, what if some border information border has important information? We cannot afford to lose it. Therefore, we use a padding concept. Now, let's take the same example three earlier. Yeah, before three was you were seeing a three this, then when you stride three is looked only one time. But now, you just see if a three is a written here, if you consider three cross three, is it not the three is looked once?

[36:36] Then, when you stride, is it not three is looked at second time? Then, when you stride it down, is it not three is again seen? When you stride, again three is seen. This was not a case before, but by padding, you can able to see the border information also. That is the main advantage of going over a padding. Now, if a padding is applied, how padding is applied? You are applying a zero in all the borders. Now, when you apply a kernel, which is a three cross three, what should be the size? The size has to be six cross six.

[37:16] If the original See, 1 2 3 4 5 6. So, the original dimension This was your original image, which was six cross six. For this, you have put one border, which is a padding, zero padding. So, the original input should be back to six cross six only. After we perform a convolution operation, the size has to be six cross six. Then, only we can say that information is not been lost. If this is the case, what is the mathematical formula? The mathematical formula is it is n plus $2p$, that is the padding, minus f

plus one. So, what was the formula earlier? It is $n - f + 1$. Now, since a padding is applied, you are writing $2p$ as well. So, now substitute the values. It is a six, n value is original six, plus you have

[38:06] applied one layer of pooling, so it is one, minus f , f size is a three, plus one. So, it is a six plus two minus three plus one. So, which is nothing but the six. Right? So, this is what is padding concept. So, I hope this is cleared to you. Now, the last concept here is a stride. What exactly is a stride? Stride is an hyper parameter here. And in Keras 2D, the stride always controls how the filter has to move over the entire input image.

[38:49] So, here this is the filter or a kernel. So, when you move when you put this kernel onto the image, so this is what is since it is 3 cross 3 you are covering 3 cross 3 part of an input image. Now, when when once you perform a convolution operation between this, you have to stride it. So, now the stride here is one. So, what is what exactly is a stride? Again, when you move So, you you will be now considering this part of an image. So, stride is a parameter which will control how the filter has to move over an input image.

[39:30] If the stride is considered, then what is the formula that is used to calculate the feature map? Here, if the padding is a considered, then write $2p$. If padding is a not considered, then it is $n - f$ divided by s plus one. Because the stride is considered, I'm uh showing a formula with the s here. Now, since s value is one, substitute the values into this formula. So, n value is seven. Here, padding is not been applied, so padding is a zero.

[40:01] F value is a three and the stride is one plus one. So, when you substitute the values into this and when we calculate, we are getting the size of 5 cross 5. So, this is what is the feature map. Now, uh uh you can ask the question, is a stride always a value one? Answer is no. You can have a stride value even a two. So, you can see here, you are uh shifting you are leaving a two positions and then you're shifting. Earlier, this was your stride. Now, you're leaving these two and then you consider this part.

[40:36] This is the stride. So, in that case, is it not S value is two here? So, again, when you apply a convolution operation, so, if you see here, you have uh shifted again uh two. You're leaving a two and then uh you're leaving these two and then you're considering this. So, this is a stride equal to two. When we apply uh a kernel, you see the output size is a three cross three. Onto this, when we apply a kernel, the output what we are getting is three cross three. How uh we are uh see that there is a reduction in the feature map.

[41:13] Substitute the values back into this. So, it is uh seven uh plus then uh there is a no padding, so zero minus kernel is a three, stride is a two plus one. So, when you substitute the uh values, so seven minus three by two plus one is a four by two plus one, which is a three. Similar way, when you do the second part, it is three cross

three. This is what is the uh answer that we have obtained. Now, you can ask the question, whenever we are doing a division operation, and uh for some size of input, for some size of the kernel, for some uh stride value, what if we get to some decimal value? Possible, right? We can have a decimal value also. Suppose, if you get a decimal value, the formula internally,

[42:03] it will consider the floor. This is a floor. So, what exactly is a floor? Suppose, if you have a decimal value something like 3.5, 3.1, 3.9, the resultant value will be three. This is what is the floor. It will always fall down to the greatest integer less than or equal to that number. This is what is the stride. Although you will see this particular slide in the hands-on implementation hands-on session, quickly let us do a revision of whatever has been discussed so far. So, whenever we are we are doing a convolution operation, whenever we want to do a convolution operation, what is that minimum that is required to us? We require an input, right? So, input here is of dimension 64 64 1.

[42:53] One indicates the number of channel. Here we are considering a grayscale image or it is a binary image where the number of channel here is one. Now, whenever you want to perform a convolution operation, you have an input image. Along with input image, what else is required to us? Kernel. Kernel has number of kernels and the size of a kernel. So, 32 says number of such kernels. 3 cross 3 indicates each kernel is of a size 3 cross 3.

[43:23] Diagrammatically, if you have to represent, you have an input image 64 cross 64. We have 32 such a kernels. 32 such kernels, which is of a size 3 cross 3. This kernel is applied onto this image, we get one feature map. This kernel is applied onto this image, one feature map. This kernel onto this, one feature map. Why? Why to use so many? The same kernel again and again when you go through this entire image, you will extract the same features, but we want a different features to be extracted. Then use different kernels. If 32 is a fixed number, it's an hyperparameter. 3 cross 3 is a fixed, it's an hyperparameter.

[44:06] You will see that in an hands-on session. So, now, in case if you want a padding to be applied, then you will be writing a padding equal to same. When you write a it's a it's a parameter that you'll be passing to an function called Conv2D because you're performing a convolution operation. We require an input image. We require a kernel number of kernels. What should be the stride? Here the stride is 2 cross 2, which means you are striding two positions.

[44:38] » [snorts] » Then, whether you want a padding to be applied or not. So, if you want a padding to be applied, padding equal to same. Suppose you don't want a padding to be applied, then you can write it as padding equal to valid. When you write padding equal to valid, it means no padding is being applied. Then, the activation function you have to use, which is a ReLU here. Which we'll be seeing in the upcoming

session. So, this is what is the hands-on implementation part. Now, when you here if you see, stride is not being mentioned. If a stride is not mentioned, by default the stride value will be one. One will be the stride value. And one more important aspect that you should consider here is

[45:27] here the input is 64 64, which means the size is the same. It's perfectly a square matrix, 64 or 64. But in a real application, you may even have an input image of different aspect ratio. What do you mean by this? You can even have an image which is of rectangular shape. In that case, you cannot have stride even stride. You can in that case, you can have a stride value different, which is you can have stride of 2,3 then 3,4.

[46:03] You can have a different aspect ratio. In that case, how does it mean? It moves three pixels right and then two pixels down. This is what is the meaning of a stride of 2,3. So, when is that you go for this? In case if you have a question, is it compulsory that always the stride should be a square one? The answer is no. That depends on what is the input that you are considering. Because here input is a square input that we have considered, the stride is also a square stride. But if an input is of different aspect ratio, stride is also of different aspect ratio.

[46:43] So, with this we have come to an end of this particular session where the main aim here is to understand the architecture of a CNN and why we have to learn CNN here is going forward, there'll be lot many topics which are based on CNN. So, if you know the CNN concept, it becomes easy for you to understand. So, in this particular session we have started with CNN model. We have seen convolutional layer in detail. In the coming session, we will look into the next components, which is a ReLU, pooling and a flatten the layer.

[47:18] Thank you.